Rewrite — ein Mathematiksystem

Handbuch

Rainer Heintzmann

1992–1996, überarbeitet 2026

Zusammenfassung

Rewrite ist ein System zur Umformung symbolischer Ausdrücke und zum Durchführen numerischer Rechnungen. Ausdrücke werden in Regelsystemen ('Algebren') umgeschrieben; so lassen sich Ableitungen, Vereinfachungen, Integrale und Lösungen von Gleichungen berechnen. Die Sprache selbst kennt keine Datentypen und keine Operatoren: Sorten, Funktionen und Regeln werden in Bibliotheken definiert. Die Standardbibliothek `inc/all` enthält die reellen, komplexen und exakten Zahlen, die elementaren und viele spezielle Funktionen, Listen und Matrizen, das Lösen von Gleichungen und eine regelbasierte Integration (eine Übersetzung der Rubi-Regeln). Numerische Funktionen sind in C geschrieben und werden mit `ATT` an Funktionen der Sprache gebunden.

Dieses Handbuch beschreibt die Sprache. Die englische Beschreibung `README.md` enthält dazu die Bibliotheken ausführlicher.

Inhaltsverzeichnis

1 Übersetzen und Starten **2**
- 1.1 Übersetzen . . . 2
- 1.2 Starten . . . 2
- 1.3 Terminal mit Zeileneditor . . . 2

2 Eingabe **3**

3 Datentypen und Operatoren **3**

4 Regelsysteme und Umschreiben **5**
- 4.1 Bedingungen . . . 6
- 4.2 Optionale Teile und kommutative Operatoren . . . 6
- 4.3 Grenzen und Lösungsschritte . . . 7

5 Term-Variablen und Zuweisungen **7**

6 Kontrollstrukturen **7**

7 Ein- und Ausgabe **8**

8 Die Standardbibliothek **8**

9 Eigene C-Funktionen **9**

10 Bekannte Einschränkungen **9**

A Die Grammatik **9**

1 Übersetzen und Starten

1.1 Übersetzen

Benötigt werden `gcc`, `flex`, `bison`, `make` und die GMP-Bibliothek (für die exakten Zahlen).

Linux: `make`, danach `make test` für die Tests.

Windows: MSYS2 installieren (z. B. mit `winget install MSYS2.MSYS2`) und in der MSYS2-Shell einmalig die Werkzeuge:

```
pacman -S mingw-w64-ucrt-x86_64-gcc mingw-w64-ucrt-x86_64-gmp flex bison make diffutils
```

Danach im Projektverzeichnis `build_windows.bat` aufrufen (`build_windows.bat test` für die Tests). Das erzeugte `rewrite.exe` braucht keine weiteren DLLs.

1.2 Starten

Starten Sie `rewrite` in der Shell im Projektverzeichnis (die Pfade in `INC` und `DATA` gelten relativ zum aktuellen Verzeichnis) und geben Sie `INC "inc/all"` ein. Dann können Sie Ihre Aufgabe stellen:

```
> INC "inc/all"
> REWR D(sin cos x,x) : All
- cos (cos (x ))* sin (x )
> REWR I(0,1,x^2,x) : numerik,calc
(1/3)
> [[1,2],[3,4]] * [5,6]
[17,
 39]
```

Das Programm bestimmt die Ableitung von $\sin(\cos x)$ nach x durch Umschreiben des Terms in der Regelliste `All`.

Aufruf	Bedeutung
`rewrite`	interaktiv: Begrüßung, dann eine Eingabeaufforderung `>` für jede Zeile (`->` in einem offenen Block)
`rewrite datei ...`	führt die Dateien aus (wie `INC "datei"`) und endet
`rewrite -i datei ...`	führt die Dateien aus und geht interaktiv weiter
`rewrite < datei`	liest das Programm aus einer Datei oder Pipe, ohne Begrüßung und Eingabeaufforderung (für Skripte und die Tests)
`rewrite -h`	zeigt die Optionen
`-no-startup`	interaktiv ohne die Startdatei
`-max-steps n`	die Schrittgrenze von `REWR` (wie `MAXSTEPS n`)
`-max-depth n`	die Tiefengrenze von `REWR` (wie `MAXDEPTH n`)

Eine interaktive Sitzung führt zuerst die Startdatei `.rewrite/startup.rw` aus, wenn es sie gibt (wie `startup.jl` in Julia). Sie wird in `HOME` gesucht, dann im Windows-Benutzerverzeichnis. Eine Zeile `INC "inc/all"` darin lädt die Bibliothek in jeder Sitzung. `QUIT` oder das Dateiende (Strg-D, unter Windows Strg-Z und Enter) beendet `rewrite`. Strg-C bricht eine laufende Rechnung ab, ohne `rewrite` zu beenden.

1.3 Terminal mit Zeileneditor

Das Skript `tools/rewrite-rl` startet `rewrite` mit `rlwrap`: Zeilen lassen sich editieren, die Eingaben bleiben über Sitzungen hinweg erhalten (`~/.rewrite_history`), die Pfeiltasten suchen wie in Julia nach früheren Zeilen mit dem gleichen Anfang, TAB ergänzt Schlüsselwörter

und Funktionsnamen, und wie in Julia wird `\alpha` mit TAB zu α. Schlüsselwörter, Funktionen und Sorten werden farbig angezeigt. Unter Windows legt `tools/make_shortcut.ps1` eine Verknüpfung auf dem Desktop an.

2 Eingabe

- Die Eingabe wird zeilenweise verarbeitet; jede Zeile wird gleich nach dem Lesen ausgeführt.
- Eine lange Zeile wird mit `\` am Zeilenende fortgesetzt. Hinter dem `\` darf kein Zeichen stehen, auch kein Leerzeichen.
- Ein Block in `{ ... }` darf über mehrere Zeilen gehen.
- Mehrere Befehle in einer Zeile werden durch `;` getrennt.
- Kommentare beginnen mit `#` und gehen bis zum Zeilenende.
- Eine Zeile, die mit `?` beginnt, zeigt eine Hilfe wie in Julia: `?REWR`, `?Int`, `?sin`; `?` allein listet alle Namen mit einem Hilfetext. Die Texte stehen in der Bibliothek (`HELP "name" "text"`, siehe `inc/help`). Ohne Text zeigt `?f` die Varianten einer Funktion und `?x` Art und Wert eines Namens.
- Ein Text in doppelten Anführungszeichen wird ausgegeben. Eine durch Kommas getrennte Liste gibt mehreres aus: `"x = ",x`.
- Ein hingeschriebener Term wird ausgegeben, so weit wie möglich ausgerechnet. Der zuletzt ausgegebene Term ist immer als Term `%` verfügbar.
- Fehler werden mit der fehlerhaften Zeile gemeldet; danach geht es mit der nächsten Zeile weiter.
- Namen dürfen Unicode-Buchstaben enthalten (UTF-8): `SYM` α`,` θ_1 `: real`. Wie in Julia werden LaTeX-artige Namen bei der Eingabe ersetzt: `\alpha` wird zu α, `\_1` zu $_1$, `\hbar` zu $\hbar$ (Julias Tabelle mit etwa 2500 Namen, `src/latex.c`). Ein Name, der auf einen Buchstaben endet, darf nicht von einem Buchstaben gefolgt sein: `\alphax` bleibt stehen.
- Unicode-Operatoren sind eigene Zeichen, auch ohne Leerzeichen: `a`$\leq$`b` ist `a` $\leq$ `b`, `3×4` ist `3 × 4`. Das sind die Zeichen $\times \div \cdot \pm \neg$, die Pfeile und die mathematischen Operatoren, außer denen, die als Namen dienen ($\partial \nabla \infty$, Summen und Integrale). Ihre Bedeutung kommt aus der Bibliothek (`ALIAS`, Abschnitt 3): `inc/alg` liest $\times$ als `*`, ebenso $\leq \geq \neq \cdot$ $\cdot \div - \wedge \vee \neg \surd \pi$ ($\surd$`16` ist `sqrt 16`).

3 Datentypen und Operatoren

Ohne Bibliothek kennt die Sprache weder Datentypen noch Operatoren. Dieses vollständige Programm definiert sie selbst:

```
SORT real                                   # ein Datentyp
REALNUM real                                # Zahlen wie 1.5 haben den Typ real
FUNC + : real,real >> real LA2 ATT plus     # Operatoren mit Rang und der
FUNC * : real,real >> real LA3 ATT mult     # C-Funktion, die sie ausrechnet
FUNC ^ : real,real >> real RA  ATT power
1+2*3                                       # gibt 7 aus
SYM x,y : real
x+y*2                                       # gibt x + y * 2 aus
x=4
x+y*2                                       # gibt 4 + y * 2 aus
```

SORT name [,name ..] Erzeugt einen Datentyp (eine 'Sorte'). Sorten werden bei Funktionen, Symbolen und Termen gebraucht.

REALNUM sortname Alle eingegebenen Zahlen (z. B. `1.234`) bekommen diese Sorte; sie werden als `double` gespeichert. Vor `REALNUM` ist eine Zahl ein Fehler. Bsp.: `SORT real REALNUM real`

INTNUM sortname ATT name Zahlen ohne Punkt und Exponent bekommen diese Sorte; die C-Funktion `name` erzeugt den Wert aus dem Text. So macht `inc/exact` sie zu exakten ganzen Zahlen (GMP).

SUBSORT a,b [ATT name] Jeder Wert der Sorte `a` ist auch einer der Sorte `b` (`integer` < `rational` < `real`); die C-Funktion wandelt einen Wert um. Eine Funktion für `real` nimmt dann auch exakte Argumente. Passen mehrere Varianten einer Funktion, so hat eine genau passende Vorrang, sonst gewinnt die mit den wenigsten Umwandlungen. Ohne `ATT` hat `b` keine eigenen Werte: so sind `number` (oberhalb von `real` und `complex`), `algebra`, `vector` und `matrix` Sorten für symbolische Ausdrücke. Regeln für `number` gelten für alle Terme darunter.

SYM name [,name ..] : sortname Deklariert ein Symbol (bis 2026 `CONST`). Es steht für sich selbst und wird mit seinem Namen ausgegeben, oder es hat einen Wert seiner Sorte (`x = 4`); es ist also eine Variable im üblichen Sinn. Ein Symbol einer Sorte ohne Werte (`number`) hat keinen Wert.

TERM name [,name ..] : sortname Deklariert eine Term-Variable, die einen ganzen Ausdruck speichert (Abschnitt 5).

PAT name [,name ..] : sortname Deklariert eine Muster-Variable (bis 2026 `VAR`). Sie steht in Regeln für einen beliebigen Term ihrer Sorte oder einer Sorte darunter. Bsp.: `PAT A : real  SPEC xyz : A+0=A`

NUMPAT name [,name ..] : sortname Deklariert eine Muster-Variable, die nur für Zahlen steht (bis 2026 `NUM`). Bsp.: `NUMPAT N : real  SPEC xyz : N+0=N`

TYPEOF term, TYPEOF f Gibt die Sorte eines Terms aus; für einen Namen auch seine Art (`SYM`, `TERM`, `PAT`, `NUMPAT`) und seinen Wert oder Term, für eine Funktion alle Varianten mit ihren Sorten.

FUNC name : sortliste >> sortname [LA1..LA5|RA|RU] [ATT ..] Erzeugt eine Funktion. Die Sortenliste gibt Anzahl und Sorten der Argumente an; ein `++` hinter der letzten Sorte steht für beliebig viele Argumente dieser Sorte. Der zweite Sortenname ist die Sorte des Ergebnisses. Das Ein- und Ausgabeformat:

LA1 bis LA5 links-assoziativer Operator mit zwei Argumenten; `LA5` bindet stärker als `LA4` usw.

RA rechts-assoziativer Operator, z. B. `a^b`.

RU rechts-unärer Operator, z. B. die Fakultät `3!`.

ohne die Funktion wird als `sin x`, `sin(x)` oder `f(x,y)` geschrieben.

Was die Funktion ausrechnet, legt `ATT` fest:

ATT name die C-Funktion `name` aus der Liste `userfunction` in `src/userfn.c`. Sie wird aufgerufen, wenn alle Argumente einen Zahlenwert haben; sonst bleibt die Funktion symbolisch. Bsp.: `FUNC sqrt : real >> real ATT r_sqrt`

ATT SYMBOLIC name eine symbolische C-Funktion aus `src/symfn.c`; sie bekommt die Argumente als Terme und gibt einen neuen Term zurück (z. B. `expand`, `coeff`, `normform`). Mit `ATT SYMBOLIC HOLD name` bekommt sie die Argumente unausgewertet.

ATT RULES algliste die Funktion wird durch Umschreiben berechnet: `f(a,b)` wird zu `REWR f(a,b) : algliste`. Ist das Ergebnis wieder ein Aufruf von `f`, so bleibt der Aufruf stehen. So ist die Ableitung `deriv` in `inc/helpers` definiert.

ATT (Argdefs) Block eine in der Sprache selbst geschriebene Funktion. `( Argdefs )` sind Definitionen der Argumente (Symbole, Terme oder Funktionen). Der Block wird ausgeführt, wenn der Term berechnet wird:

```
FUNC fma3 : real,real,real >> real ATT (SYM aa,bb : real; TERM cc : real) \
            { RETURN cc+aa*bb }
fma3(2,3,4)          # gibt 10 aus
```

RETURN term Setzt das Ergebnis einer selbst geschriebenen Funktion. Die Ausführung der Funktion wird dadurch *nicht* beendet.

OTHER name : sortliste >> sortname [ATT name] Gibt einem vorhandenen Funktionsnamen eine weitere Bedeutung für andere Argument- und Ergebnissorten. Funktionen werden anhand der Sorten ihrer Argumente unterschieden. Bsp.: `OTHER + : vector,vector >> vector`

BRA auf zu : sortliste >> sortname [ATT name] Definiert eine Funktion in Klammerschreibweise; `auf` und `zu` sind die Klammersymbole. Statt `[a,b,c]` darf man `a[b,c]` schreiben, so lassen sich Vektoren mit `v[1]` indizieren. Bei `OTHER` kennzeichnet die erste Klammer die Funktion. Bsp.: `BRA < > : vector,vector >> real` # Skalarprodukt `<a,b>`

ATT sortname Name1,Name2,Name3,Name4 Gibt einer Sorte die C-Funktionen zum Anzeigen, Vergleichen, Kopieren und Löschen ihrer Werte (aus `userfunction`). Die Vergleichsfunktion liefert 1, 0 oder -1 für $>$, $=$, $<$. Bsp.:

```
ATT complex show_complex,cmp_complex,cpy_complex,del_complex
```

SHOW f [ATT name] Terme mit der Funktion `f` werden mit der C-Funktion `name` ausgegeben; `SHOW f` allein schaltet zurück. So gibt `inc/list` einen Vektor als Spalte und eine Matrix als Gitter aus.

ALIAS "name" = "symbol" Der Name wird von nun an als das Symbol gelesen; die Ausgabe zeigt das Symbol. Bsp.: `ALIAS "×" = "*"`, dann ist `3×4` gleich 12.

SYMBOLIC "rolle" = "symbol" Die symbolischen C-Funktionen kennen die Operatoren der Bibliothek über Rollen (`"+"`, `"cos"`, `"pi"`, ...). Eine Bibliothek mit anderen Namen ordnet sie so zu: `SYMBOLIC "cos" = "kos"`. `SYMBOLIC` allein löscht die Zuordnung.

HELP "name" "text" Fügt der Hilfe zu `name` eine Zeile hinzu, die `?name` anzeigt.

4 Regelsysteme und Umschreiben

ALG name [,name ..] Definiert ein neues Regelsystem (eine 'Algebra').

SPEC alg : [IF bedingung] links = rechts [DO block] Löscht die alten Regeln von `alg` und trägt diese Regel ein.

ADDSPEC alg : [IF bedingung] links = rechts [DO block] Fügt `alg` eine Regel hinzu. Muster-Variablen dürfen auf beiden Seiten stehen. Ist `DO` mit einem Block angegeben, so wird der Block vor dem Ersetzen ausgeführt.

SPEC alg : alg1, alg2, .. `alg` steht für die Liste der Regelsysteme, die nacheinander angewandt werden. Eine Liste in Klammern, z. B. `(Linear,Compl,diff)`, wird so lange angewandt, bis sich nichts mehr ändert. Die Regelsysteme werden durch Kommas getrennt (in älteren Versionen durch `;`).

REWR term : alg [,alg ..] Schreibt den Term mit den Regelsystemen nacheinander um und gibt das Ergebnis aus. Mit `t = REWR term : alg` wird es in der Term-Variablen `t` gespeichert.

LIFT (funktionen) TO (funktionen) Überträgt alle Regeln, die für eine Menge von Funktionen und Konstanten gelten, auf eine andere. Die Funktionen werden durch die Sorten ihrer Argumente gekennzeichnet; die Argumente selbst spielen sonst keine Rolle. Als Konstanten sind auch 0 und 1 möglich. Mit diesem Trick folgen die Ringregeln aus den Gruppenregeln:

```
LIFT (Aag+Aag,Aag-Aag,-Aag,0ag) TO (Ar+Ar,Ar-Ar,-Ar,0r)
LIFT (Aag+Aag,Aag-Aag,-Aag,0ag) TO (Ar*Ar,Ar/Ar,mi(Ar),1r)
```

4.1 Bedingungen

Eine Regel gilt nur, wenn ihre Bedingungen erfüllt sind:

LB(term,term) 'lower or between': eine Ordnung der Terme, z. B. zum Sortieren von Summen: `ADDSPEC real : IF LB(A,B) B+A=A+B`.

IN(a,b) der Term `a` kommt in `b` vor. `NIN(a,b)` ist die Verneinung; `NIN(x,a,b,c)` prüft, dass `x` in keinem von `a`, `b`, `c` vorkommt.

(term) ein Ausdruck mit einem Wahrheitswert, z. B. `(is_int(N))` oder `(N > 0)`. Lässt er sich nicht ausrechnen (etwa weil `N` symbolisch ist), so gilt die Bedingung nicht.

Mehrere Bedingungen werden durch Kommas getrennt und müssen alle gelten: `IF NIN(x,A), (N > 0) ...`

```
PAT A : real
ALG einfach
SPEC einfach : A*1=A
ADDSPEC einfach : A+0=A
SYM u : real
REWR (u*1)+0 : einfach      # gibt u aus
```

4.2 Optionale Teile und kommutative Operatoren

Drei Befehle lassen eine Regel viele Formen eines Terms erfassen. Sie gelten für die danach definierten Regeln:

OPT A,B die Muster-Variablen `A`, `B` dürfen im Term fehlen.

DEFAULT + = 0 ein fehlender Operand von `+` hat den Wert 0; `DEFAULT RIGHT ^ = 1`: nur der rechte Operand von `^` darf fehlen.

COMM +,* Muster mit `+` und `*` passen auch mit vertauschten Operanden.

OPT, DEFAULT, COMM ohne Argumente: löscht die Liste wieder.

Solche Regeln stehen nicht im Regelbaum, sondern in einer geordneten Regelliste (wie in Mathematica oder Rubi): die erste passende Regel gewinnt. Der Matcher (`src/listmatch.c`) probiert vertauschte Operanden, andere Klammerungen und fehlende Operanden mit Backtracking durch; schlägt eine Bedingung fehl, so wird die nächste Möglichkeit versucht. Ein Ergebnis, das dem Term gleicht, einen NaN enthält oder einen Zyklus schließt, wird übersprungen.

4.3 Grenzen und Lösungsschritte

MAXSTEPS [n] Ein `REWR` bricht nach `n` Umschreibschritten ab (voreingestellt 100000) mit der Meldung `REWR : step limit ... reached`. Regeln, die sich im Kreis drehen, halten `rewrite` so nicht auf. Ohne `n` wird die Grenze angezeigt.

MAXDEPTH [n] Eine Regel, die den Term bei jedem Schritt tiefer macht, etwa `lin(c) = c*lin(1)` (sie passt wieder auf `lin(1)`), würde den Stapel des Interpreters lange vor der Schrittgrenze überlaufen lassen. `REWR` bricht deshalb auch ab, wenn der Term tiefer als `n` Ebenen wird (voreingestellt 1000; eine Summe von 1000 Summanden ist 1000 Ebenen tief). Eine höhere Grenze kostet bei einer solchen Regel Zeit, eine sehr hohe kann `rewrite` zum Absturz bringen.

STEPS alg "datei" Zeigt jede angewandte Regel der Regelliste von `alg` (die Lösungsschritte), beschriftet aus `datei` (Zeilen `nummer<TAB>name<TAB>text`); `STEPS alg ""` schaltet es ab.

5 Term-Variablen und Zuweisungen

termname = term Speichert den Term in der Term-Variablen. Wird eine Term-Variable einer anderen zugewiesen, so wird der Inhalt kopiert (und, wo möglich, ausgerechnet); spätere Änderungen wirken sich nicht aus. Lokale Symbole eines Blocks dürfen nicht in globale Terme gelangen.

symbol = wert Gibt einem Symbol einen Wert, z. B. `x = 1+1+3`. Lässt sich der Wert nicht ausrechnen, so verliert das Symbol seinen Wert und wird wieder mit seinem Namen ausgegeben. Das Symbol `nothing` der Bibliothek hat nie einen Wert, so löscht `a = nothing` den Wert von `a`.

Symbole und Term-Variablen werden durch Hinschreiben ihres Namens ausgegeben. Eine Term-Variable wirkt wie eine Funktion ihrer Symbole:

```
TERM tt,qq : real
tt = a+b
tt*2                # gibt (a + b )* 2 aus
%+1                 # gibt (a + b )* 2 + 1 aus
qq = x^y*a          # wie eine Funktion qq(x,y,a)
x=4
y=2
a=3
qq                  # gibt 48 aus
a = nothing
a + 1               # gibt a + 1 aus
```

6 Kontrollstrukturen

FOR symbol = n1 TO n2 [STEP n3] DO block Schleife über ein Symbol. Bsp.: `FOR n = 20 TO 10 STEP -1 DO n`

IF (bedingung) block [ELSE block] Bedingung mit einem Wahrheitswert. Bsp.: `IF (n > 3) "gross" ELSE "klein"`

WHILE (bedingung) block Schleife. Bsp.: `WHILE (n < 100) n = n*2`

{ ... } Block. Die in einem Block deklarierten Symbole sind lokal und werden bei jedem Schleifendurchlauf neu angelegt:

```
FOR n = 1 TO 3 DO { SYM s : real
                    s = n*10
                    PRT s }
```

7 Ein- und Ausgabe

INC "datei" Liest die Datei und führt sie aus. `INC` darf geschachtelt werden. Eine Zeile mit `INC` darf nicht mit \ fortgesetzt werden. Steht `INC` in einem Block, so sind die Definitionen der Datei trotzdem global.

PRT liste Gibt aus, ohne Zeilenende.

PRTNUM "format" Das C-Format für reelle Zahlen, z. B. `PRTNUM "%.10g "`.

OUT "datei" Lenkt die Ausgabe in die Datei um, die vorher gelöscht wird; `OUT "stdout"` schaltet zurück. So lassen sich Daten für Grafikprogramme erzeugen.

DATA "datei" Öffnet eine Datei zum Lesen von Zahlen.

DATA symbol Liest die nächste Zahl aus der geöffneten Datei in das Symbol; nichtnumerischer Text bis zum Zeilenende wird übersprungen. So lassen sich z. B. Messwerte weiterverarbeiten.

SYS "befehl" Führt einen Befehl der Shell aus.

DEBUG n Setzt die Debug-Stufe des Interpreters.

INFO name Art und Sorten eines Namens (ältere Form von `TYPEOF`).

QUIT Beendet `rewrite`.

8 Die Standardbibliothek

`INC "inc/all"` lädt die mathematische Bibliothek:

Datei	Inhalt
`inc/alg`	Grundsorten (`real`, `bool`, `complex`, `number`, `algebra`, Gruppe, Ring, Körper), Operatoren, elementare und spezielle Funktionen, Vereinfachung, Ableitung `D`, die Regelliste `All`, die Unicode-Operatoren
`inc/pdiff`	partielle Ableitungen
`inc/numerik`	Numerik mit `numerik,calc`: Integral `I(a,b,f,x)`, Nullstelle `newton(start,f,f',x)`, `fmin`, `fmax`
`inc/mathnum`	die Zahlenwerte von `e` und `pi`, Winkeleinheiten
`inc/integration/all`	exakte Zahlen (`inc/exact`: `integer`, `rational`), die symbolischen Hilfsfunktionen (`inc/helpers`: `deriv`, `expand`, `cancel`, ...), Integration mit `Integrate`
`inc/integration/rubi/all`	die übersetzten Rubi-Regeln, `RubiIntegrate`
`inc/list`	Listen `[a,b,c]` (Sorte `vector`) und Matrizen `[[a,b],[c,d]]` (Sorte `matrix`) mit ihrer Arithmetik, `part`, `length`, `table`, `select`, ...
`inc/linalg`	`transpose`, `dot`, `det`, `linsolve`, `inverse`, Kovektoren, symbolische Matrizen
`inc/solve`	Gleichungen: `solve(lhs == rhs, x)`, auch Systeme
`inc/simplify`	`All` endet mit der Normalform (`normform`)
`inc/help`	die Hilfetexte für `?name`

Wahlweise nach `inc/all`: `inc/rvector` und `inc/rmatrix` (numerische Vektoren und Matrizen in C), `inc/physics` (Einheiten, physikalische Konstanten, Lagrange-Formalismus), `inc/eqnsolve` (der alte Gleichungslöser). Das Verzeichnis `examples` enthält kleine Beispielprogramme.

9 Eigene C-Funktionen

Neue numerische Funktionen werden in C geschrieben und in die Liste `userfunction` in `src/userfn.c` eingetragen. Eine Funktion bekommt ein Feld von Zeigern auf ihre Argumente und gibt einen Zeiger auf ein neu angelegtes Ergebnis zurück:

```
double * r_abs (double * arg[]) { return mkreal(fabs(* arg[0])); }
```

Beispiele stehen in `real.c`, `complex.c`, `vector.c` und `matrix.c`. Mit `ATT` in einer `FUNC`- oder `OTHER`-Deklaration wird die Funktion angebunden.

Symbolische Funktionen, die auf Termen statt auf Zahlen arbeiten, stehen in der Liste `symfunction` in `src/symfn.c`. Sie bekommen die Argumente als Terme und geben einen neuen Term zurück, oder `NULL`, wenn sie nicht passen; angebunden werden sie mit `ATT SYMBOLIC name`. Den Code halten sie frei von Namen der Bibliothek: sie erkennen und bauen Terme über Rollen (`is_head(t, "+", 2)`, `fsym("cos")`), die `SYMBOLIC` zuordnet.

10 Bekannte Einschränkungen

- Bei Syntaxfehlern und beim Überdefinieren von Definitionen geht noch etwas Speicher verloren. Wiederholte Rechnungen, Umschreibungen und Schleifen laufen ohne Speicherverlust.
- Nicht jede Regel, die man erwarten könnte, ist vorhanden; so werden `ln(exp x)` und $\sin^2 x + \cos^2 x$ nicht vereinfacht.
- Symbole ohne Wert dürfen von jeder Variablen abhängen, deshalb bleibt `D(a,x)` im Ergebnis stehen (gewollt: Differentialgleichungen benutzen `D(y,x)`). `deriv` behandelt dagegen jeden Term ohne `x` als Konstante.
- Das kommutative Matching vertauscht nur die zwei Operanden eines Operators; Summen und Produkte sind binäre Bäume, deren Summanden nicht über Ebenen hinweg umgeordnet werden. `COMM *` gilt auch für Regeln mit Matrixprodukten; `COMM` allein schaltet es nach den Regeln, die es brauchen, wieder ab.

A Die Grammatik

Die Zusammenfassung der Grammatik (`src/syn.y`, ohne die Aktionen in C); `python tools/gen_syntax.py` erzeugt sie neu.

```
%token SYM PAT SORT FUNC ALG SPEC ADDSPEC REWR PRT GRAPH PFEIL TERM NUMPAT ATT
%token   IS EQ INFO
%token LA1 LA2 LA3 LA4 LA5 LOCAL RU RA COMMU LB IN NIN PARAM FOR TO STEP DO DBG
%token   LAST
%token BRA OTHER DATA OUT INC EXE VARPAR SYS IF ELSE WHILE PRTNUM LIFT QUIT
%token   RETURN
%token OPT COMM DEFAULT RIGHT SUBSORT INTNUM STEPS MAXSTEPS MAXDEPTH SYMBOLIC
%token   HOLD RULES SHOW REALNUM TYPEOF HELP ALIAS
%token <y_sym> Id String
%token <y_sym> Sort Alg Term
```

```
%token <y_sym> La1 La2 La3 La4 La5 Ru Ra Func Const Var Bra Bra1 Bra2
%token Terms Form Sorts Functs Forms Algs Cond2 Cond
%type <y_int> PRECEDENCE
%type <y_nod> vinstr definition instruction aninstruction instructions prntexpr
%type    tinstructions oinstructions
%type <y_nod> idlist l_thing l_list sortlist vsortlist funclist formlist
%type    termlist alglist Alg2 prntlist
%type <y_nod> term func step att namedId d_list
%type <y_nod> formula cformula condition condfunc
%left La1
%left La2
%left La3
%left La4
%left La5
%right Ra
%right Ru
%left Func
%%

prog : line
        | prog line

line : instructions '\n'

definition : FUNC namedId ':' vsortlist PFEIL Sort PRECEDENCE att
        | OTHER func ':' vsortlist PFEIL Sort att
        | ATT Sort Id ',' Id ',' Id ',' Id
        | BRA Id Id ':' vsortlist PFEIL Sort att
        | PAT idlist ':' Sort
        | REALNUM Sort
        | SUBSORT Sort ',' Sort ATT Id
        | SUBSORT Sort ',' Sort
        | SHOW func ATT Id
        | SHOW func
        | INTNUM Sort ATT Id
        | NUMPAT idlist ':' Sort
        | SYM idlist ':' Sort
        | TYPEOF term
        | TYPEOF func
        | HELP String String
        | SPEC Alg ':' alglist
        | SPEC Alg ':' formlist
        | ADDSPEC Alg ':' formlist
        | SORT idlist
        | ALG idlist
        | TERM idlist ':' Sort

idlist : namedId
        | namedId ',' idlist

l_thing : Const
        | Term

l_list : l_thing
        | l_thing ',' l_list

att :
        | ATT namedId
```

```
        | ATT SYMBOLIC namedId
        | ATT RULES alglist
        | ATT SYMBOLIC HOLD namedId
        | ATT '(' d_list ')' instruction

namedId : Id
        | Const
        | Var
        | func
        | Sort
        | Alg
        | Term

PRECEDENCE :
        | LA1
        | LA2
        | LA3
        | LA4
        | LA5
        | RA
        | RU

condition : condfunc
        | condfunc ',' condition

condfunc : LB '(' termlist ')'
        | IN '(' termlist ')'
        | NIN '(' termlist ')'
        | '(' term ')'

instructions : instruction
        | instruction ';' instructions

tinstructions : oinstructions
        | oinstructions '\n' tinstructions

oinstructions : instructions

separator : '\n'
        | ';'

d_list : definition
        | definition ';' d_list

instruction : aninstruction
        | definition
        | '{' tinstructions '}'
        | WHILE condition instruction
        | IF condition instruction
        | IF condition instruction ELSE instruction
        | FOR term EQ term TO term step DO instruction

aninstruction :
        | SYS String
        | INC String
        | INFO namedId
        | REWR term ':' alglist
        | prntlist
```

```
        | PRTNUM String
        | PRT prntlist
        | OUT String
        | DATA String
        | DATA Const
        | Term EQ term
        | Term EQ REWR term ':' alglist
        | Const EQ term
        | EXE func Id
        | LIFT '(' termlist ')' TO '(' termlist ')'
        | DBG Const
        | MAXSTEPS Const
        | MAXSTEPS
        | MAXDEPTH Const
        | MAXDEPTH
        | STEPS Alg String
        | SYMBOLIC String EQ String
        | SYMBOLIC
        | ALIAS String EQ String
        | RETURN term
        | QUIT
        | OPT idlist
        | OPT
        | COMM funclist
        | COMM
        | DEFAULT
        | DEFAULT func EQ term
        | DEFAULT RIGHT func EQ term

step :
        | STEP term

prntexpr : term
        | String

prntlist : prntexpr
        | prntexpr ',' prntlist

alglist : Alg2
        | Alg2 ',' alglist

Alg2 : Alg
        | '(' alglist ')'

sortlist : Sort
        | Sort ',' sortlist

vsortlist : sortlist
        | sortlist VARPAR

func : Func
        | La1
        | La2
        | La3
        | La4
        | La5
        | Ra
        | Ru
```

```
        | Bra
        | Bra1

funclist : func
        | func ',' funclist

term : Term
        | LAST
        | Const
        | Const Bra1 termlist Bra2
        | Var
        | '(' term ')'
        | term Ru
        | Func term
        | La1 term
        | La2 term
        | La3 term
        | La4 term
        | La5 term
        | Bra termlist Bra
        | Bra1 termlist Bra2
        | Func '(' termlist ')'
        | term La1 term
        | term La2 term
        | term La3 term
        | term La4 term
        | term La5 term
        | term Ra term

termlist : term
        | term ',' termlist

formula : term EQ term

vinstr :
        | DO instruction

cformula : formula vinstr
        | IF condition formula vinstr

formlist : cformula

%%
```